

Good afternoon, everyone.

Today, we would like to present our Explainable AI course project, titled *“From Accuracy to Explainability: Understanding CNN Errors with xAI.”*

In this project, we focus not only on how well a CNN performs, but more importantly, on how and why it makes mistakes.

This project was conducted as part of the Explainable AI course. Like most other teams, this was our first time building a CNN model and combining it with explainability methods. At first, our intuition was simple: if the accuracy looks good, the model should be reliable. However, we quickly realized that accuracy alone does not tell us how the model behaves in critical cases. Especially in real-world scenarios, confident but wrong predictions can be risky. This motivated us to look beyond standard performance metrics and to focus on explainability.

Based on this motivation, we formulated three guiding questions.

- 1) **First**, what kinds of errors does a CNN make on a realistic image classification task?
- 2) **Second**, how confident is the model when it is wrong?
- 3) Third, can explainable AI methods help us understand the underlying reasons for these errors?

The red thread of our project follows four steps:

We first **build** a baseline model,

then **observe** its behaviour,

question surprising patterns,

and finally use **explainability** methods to interpret them.

This structure guided both our analysis and our presentation.

Before going into our experiments, I would like to briefly explain the basic idea of convolutional neural networks from a learner's perspective. CNN's are designed to process image data by learning hierarchical visual features. In early layers, the model typically learns simple patterns such as edges or textures. As we move deeper into the network, these features are combined into more complex shapes and object parts. This hierarchical structure makes CNN's especially suitable for image classification tasks. Instead of manually defining features, the network learns them automatically from data. Instead, we wanted a model that is simple enough to understand, but powerful enough to show meaningful behaviour. This is important because our focus is not only on performance but on understanding how the model makes decisions.

Based on this idea, we chose ResNet18 as our backbone model.

ResNet is a well-known CNN architecture that uses skip connections to stabilize training and improve performance.

Our task is a 10-class image classification problem using an ImageNet subset with everyday objects.

To better understand the influence of different design choices, we built several baseline models.

These models differ in their pretraining source and training strategy. The goal of this setup was not to achieve the highest possible accuracy, but to provide a controlled, understandable starting point for later analysis. **(A adds briefly:)**

This baseline design allows us to compare models systematically and focus on interpreting their behaviour rather than optimizing hyperparameters.